

assignment7

DSA Assignment #7: 🌲 (2/3)

Updated 2026-04-20 16:26 GMT+8

Compiled by 徐前 物理学院 (2026 Spring)

说明:

1. 解题与记录:

对于每一个题目，请提供其解题思路（可选），并附上使用Python或C++编写的源代码（确保已在OpenJudge, Codeforces, LeetCode等平台上获得Accepted）。请将这些信息连同显示“Accepted”的截图一起填写到下方的作业模板中。（推荐使用Typora <https://typoraio.cn> 进行编辑，当然你也可以选择Word。）无论题目是否已通过，请标明每个题目大致花费的时间。

2. **提交安排:** 提交时，请首先上传PDF格式的文件，并将.md或.doc格式的文件作为附件上传至右侧的“作业评论”区。确保你的Canvas账户有一个清晰可见的本人头像，提交的文件为PDF格式，并且“作业评论”区包含上传的.md或.doc附件。

3. **延迟提交:** 如果你预计无法在截止日期前提交作业，请提前告知具体原因。这有助于我们了解情况并可能为你提供适当的延期或其他帮助。

请按照上述指导认真准备和提交作业，以保证顺利完成课程要求。

1. 题目

M297.二叉树的序列化与反序列化

dfs, bfs, <https://leetcode.cn/problems/serialize-and-deserialize-binary-tree/>

思路:

序列化使用BFS进行层序遍历，反序列化则是对遍历序列再做一次BFS来建树。

代码:

```
# Definition for a binary tree node.
# class TreeNode(object):
#     def __init__(self, x):
#         self.val = x
#         self.left = None
#         self.right = None
```

```

class Codec:
    def serialize(self, root):
        """Encodes a tree to a single string.

        :type root: TreeNode
        :rtype: str
        """
        q = deque([root])
        ans = []
        while q:
            n = len(q)
            for i in range(n):
                node = q.popleft()
                if node:
                    ans.append(str(node.val))
                    q.append(node.left)
                    q.append(node.right)
                else:
                    ans.append('n')
            return ' '.join(ans)

    def deserialize(self, data):
        """Decodes your encoded data to tree.

        :type data: str
        :rtype: TreeNode
        """
        s = data.split()
        if s[0] == 'n':
            return None
        root = TreeNode(int(s[0]))
        q = deque([root])
        i = 1
        n = len(s)
        while i < n:
            m = len(q)
            for _ in range(m):
                node = q.popleft()
                left = TreeNode(int(s[i])) if s[i] != 'n' else None
                i += 1
                right = TreeNode(int(s[i])) if s[i] != 'n' else None
                i += 1
                node.left = left
                node.right = right
                if left:
                    q.append(left)

```

```

        if right:
            q.append(right)
    return root

```

```

# Your Codec object will be instantiated and called as such:
# ser = Codec()
# deser = Codec()
# ans = deser.deserialize(ser.serialize(root))

```

代码运行截图（至少包含有"Accepted"）

通过 53 / 53 个通过的测试用例

Jolly Turing00u 提交于 2026.04.09 15:49

分析

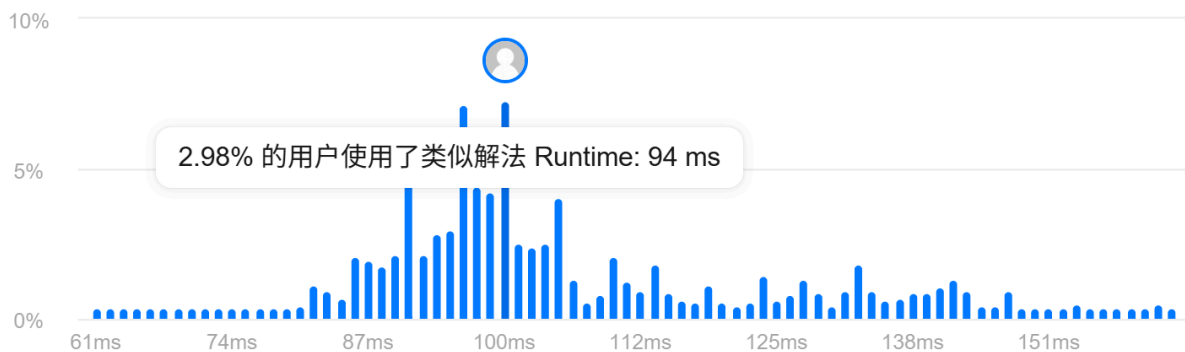
写题解

🕒 执行用时分布

100 ms | 击败 54.40% 🏆

💾 消耗内存分布

21.93 MB | 击败 92.00% 🏆



M129.求根节点到叶节点数字之和

dfs, <https://leetcode.cn/problems/sum-root-to-leaf-numbers/>

思路：

正常 dfs 遍历，记录路径上的数字。

代码：

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def sumNumbers(self, root: Optional[TreeNode]) -> int:
        def dfs(node, s):

```

```

flag = 0
sn = s * 10 + node.val
if node.left:
    flag = 1
    dfs(node.left, sn)
if node.right:
    flag = 1
    dfs(node.right, sn)
if not flag:
    nonlocal ans
    ans += sn
ans = 0
dfs(root, 0)
return ans

```

代码运行截图 (至少包含有"Accepted")

通过 108 / 108 个通过的测试用例

Jolly Turing00u 提交于 2026.04.14 17:45

分析

写题解



面向在校学生的专享特惠

完成认证享 7 折 Plus 会员，享受更多学业及职业成长帮助

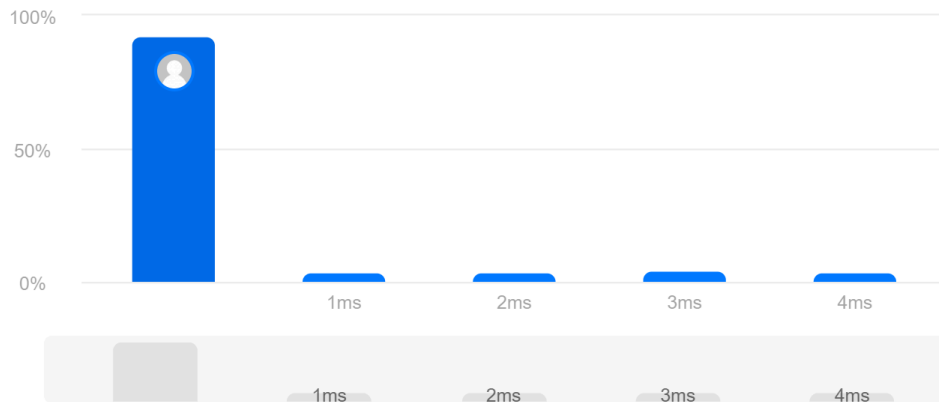


执行用时分布

0 ms | 击败 100.00%

消耗内存分布

19.07 MB | 击败 70.96%



```

1 # Definition f
2 # class TreeNo
3 #     def __in
4 #         self
5 #         self
6 #         self
7 class Solution
8     def sumNum
9         def df
10             fl
11             sn
12             if
13             |
14             |
15             if
16             |
17             |
18             if
19

```

已存储

测试用例 | 测试

通过 执行用时: 0 ms

Case 1



M22158:根据二叉树前中序序列建树

tree, <http://cs101.openjudge.cn/practice/22158/>

思路:

根据前序遍历找到根节点 root，然后根据 root 在中序序列中的位置划分左右子树，然后递归求解子树。

代码：

```
def pre_in_to_post(s_pre, s_in) -> str:
    root = s_pre[0]
    index = s_in.index(root)
    l1 = index
    l2 = len(s_in) - index - 1
    ans = ''
    if l1:
        ans = ans + pre_in_to_post(s_pre[1:l1+1], s_in[:index])
    if l2:
        ans = ans + pre_in_to_post(s_pre[l1+1:], s_in[index+1:])
    return ans + root
while True:
    try:
        s_pre = input()
        s_in = input()
        print(pre_in_to_post(s_pre, s_in))
    except EOFError:
        break
```

代码运行截图（至少包含有"Accepted"）

#52300495提交状态

[查看](#) [提交](#) [统计](#) [提问](#)

状态: Accepted

源代码

```
def pre_in_to_post(s_pre, s_in) -> str:
    root = s_pre[0]
    index = s_in.index(root)
    l1 = index
    l2 = len(s_in) - index - 1
    ans = ''
    if l1:
        ans = ans + pre_in_to_post(s_pre[1:l1+1], s_in[:index])
    if l2:
        ans = ans + pre_in_to_post(s_pre[l1+1:], s_in[index+1:])
    return ans + root
while True:
    try:
        s_pre = input()
        s_in = input()
        print(pre_in_to_post(s_pre, s_in))
    except EOFError:
        break
```

基本信息

#: 52300495
题目: 22158
提交人: 25n2500011619(xq)
内存: 3632kB
时间: 30ms
语言: Python3
提交时间: 2026-04-14 21:20:16

M24729:括号嵌套树

dfs, stack, <http://cs101.openjudge.cn/practice/24729/>

思路：

多叉树用邻接表实现边的存储，递归构建树结构，每次遇到字母创建节点，遇到（递归遍历子节点，遇到）返回节点表示子树结束。

代码：

```
class TreeNode:
    __slots__ = ('val', 'children')
    def __init__(self, val):
        self.val = val
        self.children = []

def parse_tree(s) -> TreeNode:
    i = 0
    n = len(s)
    def parse():
        nonlocal i
        if i == n:
            return None
        node = TreeNode(s[i]) ## 创建新节点
        i += 1
        if i < n and s[i] == '(':
            i += 1 ## 跳过 '('
            while i < n:
                node.children.append(parse())
                if i < n and s[i] == ',':
                    i += 1 ## 下一个子节点
                elif i < n and s[i] == ')':
                    i += 1 ## 子树遍历完成，退出循环
                    break
            return node
    return parse()

def preorder(node):
    res = [node.val]
    for child in node.children:
        res.extend(preorder(child))
    return ''.join(res)

def postorder(node):
    res = []
    #print(' '*(4*x)+node.val)
    for child in node.children:
        res.extend(postorder(child))
```

```

        return ''.join(res) + node.val

def main():
    s = input().strip()
    tree = parse_tree(s)
    print(preorder(tree))
    print(postorder(tree))

if __name__ == '__main__':
    main()

```

代码运行截图（至少包含有"Accepted"）

#52300914提交状态

[查看](#) [提交](#) [统计](#) [提问](#)

状态: **Accepted**

源代码

```

class TreeNode:
    __slots__ = ('val', 'children')
    def __init__(self, val):
        self.val = val
        self.children = []

def parse_tree(s) -> TreeNode:
    i = 0
    n = len(s)
    def parse():
        nonlocal i
        if i == n:
            return None
        node = TreeNode(s[i])
        i += 1
        if i < n and s[i] == '(':
            i += 1

```

基本信息

#: 52300914
 题目: 24729
 提交人: 25n2500011619(xq)
 内存: 3688kB
 时间: 30ms
 语言: Python3
 提交时间: 2026-04-14 22:02:58

M01577: Falling Leaves

tree, <http://cs101.openjudge.cn/practice/01577/>

思路:

递归实现，每一次取最后一个删除的节点为root根节点，然后根据root节点的字典序将子树分成左子树和右子树，递归还原左右子树。

代码

```

def preorder(s):
    ans = []
    def dfs(seq):
        root = seq[0][0]
        ans.append(root)
        od_r = ord(root)
        left = []

```

```

        right = []
        for c in seq[1:]:
            for i in range(len(c)):
                if ord(c[i]) > od_r:
                    if i:
                        left.append(c[:i]+'a')
                    if c[i] != 'a':
                        right.append(c[i:])
                    break
            if left: dfs(left)
            if right: dfs(right)

    dfs(s)
    print(''.join(ans))

s = []
while True:
    s0 = input().strip()
    if s0 == '$':
        preorder(s[:-1])
        break
    elif s0 == '*':
        preorder(s[:-1])
        s.clear()
    else:
        s.append(s0+'a')

```

(至少包含有"Accepted")

#52316761提交状态

[查看](#) [提交](#) [统计](#) [提问](#)

状态: **Accepted**

源代码

```

def preorder(s):
    ans = []
    def dfs(seq):
        root = seq[0][0]
        ans.append(root)
        od_r = ord(root)
        left = []
        right = []
        for c in seq[1:]:
            for i in range(len(c)):
                if ord(c[i]) > od_r:
                    if i:
                        left.append(c[:i]+'a')
                    if c[i] != 'a':
                        right.append(c[i:])

```

基本信息

#: 52316761
 题目: 01577
 提交人: 25n2500011619(xq)
 内存: 3680kB
 时间: 30ms
 语言: Python3
 提交时间: 2026-04-16 21:10:03

1843D. Apple Tree

Combinatorics, dfs and similar, dp, math, trees, 1200,

<https://codeforces.com/problemset/problem/1843/D>

思路:

统计每个节点的子树中的叶子节点个数，在树上做DP。

代码

```
import sys
from collections import deque
def main():
    def dfs(n) -> None:
        fa = [0] * (n + 1)
        d = [0] * (n + 1)
        st = [(1, 0)]
        q = deque()
        while st:
            cur, pre = st.pop()
            fa[cur] = pre
            flag = True
            for nxt in edges[cur]:
                if nxt != pre:
                    flag = False
                    d[cur] += 1
                    st.append((nxt, cur))
            if flag:
                dp[cur] = 1
                q.append(cur)
        while q:
            cur = q.popleft()
            if cur == 1: break
            nxt = fa[cur]
            dp[nxt] += dp[cur]
            d[nxt] -= 1
            if d[nxt] == 0:
                q.append(nxt)

    it = iter(sys.stdin.read().split())
    t = int(next(it))
    ans = []
    for _ in range(t):
        n = int(next(it))
        edges = [[] for __ in range(n + 1)]
        dp = [0 for __ in range(n + 1)]
        for i in range(n - 1):
```

```

        u, v = int(next(it)), int(next(it))
        edges[u].append(v)
        edges[v].append(u)
    dfs(n)
    q = int(next(it))
    for i in range(q):
        x, y = int(next(it)), int(next(it))
        ans.append(dp[x] * dp[y])
    sys.stdout.write('\n'.join(map(str, ans)))

if __name__ == '__main__':
    main()

```

(至少包含有"Accepted")

By sam_xq, contest: Codeforces Round 881 (Div. 3), problem: (D) Apple Tree, **Accepted**, #, [Copy](#)

```

import sys
from collections import deque
def main():
    def dfs(n) -> None:
        fa = [0] * (n + 1)
        d = [0] * (n + 1)
        st = [(1, 0)]
        q = deque()
        while st:
            cur, pre = st.pop()
            fa[cur] = pre
            flag = True
            for nxt in edges[cur]:
                if nxt != pre:
                    flag = False
                    d[cur] += 1
                    st.append((nxt, cur))
            if flag:
                dp[cur] = 1
                q.append(cur)
        while q:
            cur = q.popleft()
            if cur == 1: break
            nxt = fa[cur]
            dp[nxt] += dp[cur]
            d[nxt] -= 1
            if d[nxt] == 0:
                q.append(nxt)

    it = iter(sys.stdin.read().split())
    t = int(next(it))
    ans = []
    for _ in range(t):
        n = int(next(it))
        edges = [[] for _ in range(n + 1)]
        dn = [0 for _ in range(n + 1)]

```

2. 学习总结和个人收获

如果发现作业题目相对简单，有否寻找额外的练习题目，如“数算2026spring每日选做”、LeetCode、Codeforces、洛谷等网站上的题目。

本周作业有许多值得学习注意的地方，树的表示法种类很多，可以根据记录的方式将其分为BFS记录和DFS记录，比如层序遍历按照BFS进队的顺序记录节点，可以用null表示空节点或者记录

度数（作业6 M07161）来标记子树结束的位置。括号表示法则按照我们DFS遍历的顺序记录节点，同样用 * 代替空节点，用) 标记子树结束，解析的时候就需要还原DFS遍历的顺序，用递归或者 **stack** 实现（1843D Apple Tree 用递归会爆栈RE，需要手写栈）。

M24729:括号嵌套树有点绕，讲义中**27637 括号嵌套二叉树**给的做法使用了全局指针加递归的方法写，但是这道题又涉及多叉树，需要用 while 循环找下一个子节点，遇到括号 (或逗号 , 需要将指针加一。相对用栈的写法比较清晰，注意判断) 对应栈顶元素处理完成，弹出栈顶节点。

学习了LCA算法的倍增法，做了LC1483 树节点的第 K 个祖先 <https://leetcode.cn/problems/kth-ancestor-of-a-tree-node/>，以及洛谷的P3379 最近公共祖先 <https://www.luogu.com.cn/problem/P3379>。洛谷的时间限制比较严，用OOP写法的倍增法代码开pypy也过不了，G老师解释局部变量的加载（LOAD_FAST）会比对象属性（LOAD_ATTR，如 self.fa）快很多。了解了离线处理LCA的Tarjan算法，需要使用并查集在DFS遍历的过程中处理LCA，可以用线性时间解决，等期中考完找时间练习一下。

代码：

```
class TreeAncestor:

    def __init__(self, n: int, parent: List[int]):
        self.n = n
        self.fa = [parent]
        i = 1
        while (1 << i) < n:
            last = self.fa[-1]
            tmp = [-1]
            for j in range(1, n):
                if last[j] == -1:
                    tmp.append(-1)
                else:
                    tmp.append(last[last[j]])
            self.fa.append(tmp)
            i += 1

    def getKthAncestor(self, node: int, k: int) -> int:
        if k > self.n - 1:
            return -1
        i = 0
        while k and node >= 0:
            if k & 1:
                node = self.fa[i][node]
            k >>= 1
            i += 1
        return node
```

```
# Your TreeAncestor object will be instantiated and called as such:
# obj = TreeAncestor(n, parent)
# param_1 = obj.getKthAncestor(node,k)
```

LCA OOP写法 (洛谷P3379):

```
from typing import List
import sys

class TreeAncestor:

    def __init__(self, n: int, root: int, edge: List[List[int]]) -> None:
        def build_tree(): # 用栈模拟实现
            st = [(root, 0)]
            while st:
                node, pre = st.pop()
                self.depth[node] = self.depth[pre] + 1
                parent[node] = pre
                for child in edge[node]:
                    if child != pre:
                        st.append((child, node))

        parent = [0] * (n + 1)
        self.depth = [0] * (n + 1)
        self.n = n
        self.fa = [parent]
        build_tree()
        i = 1
        while (1 << i) < n:
            last = self.fa[-1]
            tmp = [0]
            for j in range(1, n + 1):
                if last[j] == 0:
                    tmp.append(0)
                else:
                    tmp.append(last[last[j]])
            self.fa.append(tmp)
            i += 1

    def get_lca(self, x: int, y: int) -> int:
        if self.depth[x] < self.depth[y]:
            x, y = y, x
        d = self.depth[x] - self.depth[y]

        i = 0
```

```

        while d > 0:
            if d & 1:
                x = self.fa[i][x]
            d >>= 1
            i += 1
        if x == y:
            return x
        i = len(self.fa) - 1
        while i >= 0:
            if self.fa[i][x] != self.fa[i][y]:
                x = self.fa[i][x]
                y = self.fa[i][y]
            i -= 1
        return self.fa[0][x]

def main():
    it = iter(sys.stdin.read().split())
    n = int(next(it))
    m = int(next(it))
    root = int(next(it))
    edge = [[] for _ in range(n + 1)]
    for _ in range(n - 1):
        x = int(next(it))
        y = int(next(it))
        edge[x].append(y)
        edge[y].append(x)
    tree = TreeAncestor(n, root, edge)
    for _ in range(m):
        x = int(next(it))
        y = int(next(it))
        sys.stdout.write(str(tree.get_lca(x, y)) + '\n')

if __name__ == '__main__':
    main()

```